

RESEARCH

Open Access



Ransomware detection based on server-side file operation logs using machine learning

Gábor Arányi^{1*†}, Tamás Miseta^{1†} and Veronika Szücs^{1*†}

Abstract

Purpose This research examines the feasibility and effectiveness of detecting ransomware attacks in quasi real-time by leveraging AI-based monitoring of centralized file operations. As ransomware continues to evolve in speed and complexity, traditional endpoint protection mechanisms often fall short, especially in environments with limited client-side defense. The goal is to determine whether lightweight, server-side monitoring combined with machine learning can provide a quasi real-time and accurate detection mechanism without relying on client instrumentation.

Applied methodology A virtualized SME (small- and medium-sized enterprise) infrastructure was developed, simulating realistic user behavior through automated file operations and randomly triggered attacks by ransomware samples (Ryuk, NotPetya, Lockbit, Teslacrypt, and WannaCry). A nanosecond-scale time-stamped logging mechanism was implemented using Fluentbit and InfluxDB to track file creation, renaming, and deletion events. Five classic and ensemble machine learning models (Random Forest, Decision Tree, SVM, AdaBoost, XGBoost) were trained and optimized using supervised learning on aggregated file operation sequences using one-second intervals.

Results The comparative evaluation of the models showed that all five achieved reliable detection performance, but XGBoost outperformed the others with a sensitivity of 91.87% and prediction speeds below 1 ms. The model identified ransomware activity during the early phases of execution in the majority of test cases, even when operating in a high-noise environment with real-world file usage patterns.

Conclusions Monitoring centralized file activity offers a practical and efficient means for detecting ransomware attacks without requiring access to client systems or process-level telemetry. By using only three simple file operation metrics and binary classification, the system does not require complex, resource-intensive behavioral models.

Contribution This study presents a scalable, quasi real-time detection framework that complements existing security layers and is especially valuable in scenarios where endpoint protection is weak or inconsistent. The findings highlight an alternative direction in ransomware defense that emphasizes simplicity, performance, and deployability.

Keywords Ransomware detection, Machine learning, File operation monitoring, Real-time security, SME cyber defense

1 Introduction

At present, ransomware remains one of the most prevalent forms of cybercrime worldwide. Due to its rapid proliferation and continuous technological advances, it is important to introduce conceptual changes in defensive strategies, develop new security mechanisms, and implement multi-factor detection systems.

Recent threat intelligence consistently confirms the continued prominence of ransomware as a major cyber threat. The ENISA Threat Landscape 2024

[†]Gábor Arányi, Tamás Miseta and Veronika Szücs contributed equally to this work.

*Correspondence:

Gábor Arányi
aranyi.gabor@phd.mik.uni-pannon.hu
Veronika Szücs
szucs.veronika@mik.uni-pannon.hu

¹ Department of Electrical Engineering and Information Systems,
University of Pannonia, 10 Egyetem str., Veszprém 8200, Hungary

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

identifies ransomware among the most disruptive attack types [1], while the Verizon DBIR 2025 reports its involvement in 44% of confirmed breaches [2]. Sector-specific analyses, such as the ENISA Finance Threat Landscape 2024, highlight its persistent impact across critical industries [3]. Europol's Internet Organized Crime Threat Assessment (IOCTA) likewise identifies ransomware as one of the most prominent and disruptive forms of cybercrime in recent years [4]. These independent reports collectively underscore the scale and persistence of the threat, motivating research into more effective detection strategies.

Currently, there are many antivirus solutions and endpoint protection technologies available. However, in many cases these can be bypassed or disabled, which makes the success of an attack more likely. As such malware primarily encrypts data and renders it inaccessible, the day-to-day functioning of organizations can be partially or completely disrupted during the restoration process. To minimize the damage, it is crucial to detect such attacks as early as possible. For example, the LockBit malware was observed to encrypt 53.83 GB of data (100,000 files) across different MS Windows workstations in 4 min and 9 s, corresponding to an average speed of 216.18 MB/s. Although encryption speed is strongly influenced by factors such as network bandwidth, storage technology, interface type, encryption algorithm, and file system, these figures clearly highlight that rapid, preferably near real-time detection is essential for effective protection.

The targeted data assets are typically stored centrally and accessed by clients, often via network shares. While client-side protection is generally considered less reliable, server-side file operation logs provide direct and trustworthy information on the performed activities. Consequently, server-side monitoring can also serve as a basis for attack detection. Ransomware families tend to create, rename, and delete files with varying frequency, depending on their type. Assuming a properly designed and implemented logging system, these characteristic operation sequences can be detected at fine temporal granularity.

In real-time systems, acceptable response times are typically in the millisecond or sub-second range, depending on the application domain, and one-second latency is generally considered long. In this study, real-time operation is therefore interpreted as system behavior where event acquisition, data transfer, and model inference are all performed within millisecond-scale time frames, while higher-level analysis may rely on short aggregation windows.

In this paper, quasi real-time refers to a processing strategy where millisecond-scale events and predictions

are aggregated into one-second time windows for robust analysis, while preserving system behavior that approximates real-time operation.

In the present research, a typical SME environment was modeled in which ransomware attacks were simulated to train and evaluate machine learning algorithms for attack detection. On the created virtual infrastructure, file operations of four distinct user profiles were modeled, and five different ransomware families (Ryuk, WannaCry, NotPetya, Lockbit, and TeslaCrypt) were executed at random times. During logging, file operations on the server were recorded with nanosecond timestamps, and the number of these operations was aggregated per second. Classical and ensemble machine learning algorithms were subsequently employed to identify attacks. The primary objective of the research was to investigate whether the best performing model, after hyperparameter tuning, could detect ransomware attacks in quasi real-time with an accuracy above 90%.

Although file system events are recorded with millisecond or finer temporal resolution and model prediction is performed within milliseconds, event features are aggregated into one-second intervals. This aggregation does not reflect system latency limitations but represents a practical design decision to structure event streams into fixed temporal units for feature extraction and experimental evaluation.

A key question for enhancing effectiveness was the timely responsiveness of safeguarding mechanisms. Specifically, our research aimed to address whether an AI-based protection system can react in real or near real-time to ransomware attacks (RQ-1), and whether there exists a data collection method that enables AI-based protection to operate in near real-time (RQ-2). An additional question was whether such a solution could prevent data loss on the protected server in near real-time, independently of the specific ransomware family (RQ-3).

1.1 Scientific and practical contributions

The contributions of this study are twofold:

This work demonstrates that ransomware detection can be achieved effectively by leveraging server-side file operation logs with simple, aggregated features. We systematically compared five classical and ensemble machine learning models (RF, SVM, DT, ADA, and XGB), showing that ensemble-based approaches, particularly XGBoost, provide the most favorable relationship between sensitivity, specificity, and computational cost. The evaluation framework integrates latency measurement, ROC and precision–recall point analysis, and cost-sensitive evaluation, thereby offering a formal and reproducible methodology for benchmarking ransomware detection approaches. Furthermore, we

show that detection performance does not depend on the specific ransomware family executed, supporting the generalizability of the method under the evaluated conditions.

We propose a lightweight and easily deployable architecture that requires minimal computational overhead and can be integrated into existing SIEM or EDR infrastructures. The approach supports quasi real-time detection without intrusive kernel-level modifications and demonstrates robustness under naturally imbalanced data conditions, reflecting realistic deployment scenarios. The design does not rely on ransomware-specific signatures or obfuscation techniques, supporting practical applicability in both SME and enterprise-scale environments. Overall, the proposed framework represents a resource-efficient alternative to traditional signature-based defenses under the evaluated conditions.

2 Related works

2.1 Overview of anti-ransomware solutions

Based on the reviewed ransomware detection studies, several key challenges and potential solutions have been identified. One of the main challenges is the detection of zero-day attacks, as traditional signature-based methods cannot reliably identify new or previously unseen ransomware variants. Behavioral monitoring approaches, particularly those relying on API (Application Programming Interface) call tracing and system process analysis, often require substantial computational resources, which limits their applicability in real-time scenarios. Earlier research has also highlighted limitations of honeypot-based systems, as many solutions rely on static honeypot files that may be less effective against more sophisticated ransomware variants. Adversaries increasingly employ advanced cryptographic techniques, such as hybrid encryption, which makes file recovery highly challenging. Similarly, network traffic analysis may be of limited effectiveness, as attackers employ anonymization and money laundering techniques (e.g., mixer services) to conceal Bitcoin transactions.

Researchers have explored various approaches to address these limitations. Machine learning (ML)-based methods that rely on the analysis of behavioral patterns show promise in improving ransomware detection rates. Behavioral analysis combined with ML algorithms, such as Long Short-Term Memory (LSTM), Support Vector Machine (SVM), Random Forest, and Feed Forward Deep Neural Networks, has the potential to accelerate the identification of malicious activity [5, 6].

Decoy systems, particularly those employing dynamically generated decoy files and Moving Target Defense (MTD) techniques, also play an important role, as they

continuously modify filenames and formats to disrupt ransomware operations. To strengthen resilience against zero-day attacks, researchers have suggested continuous monitoring of file entropy and hash values, as well as pattern analysis of Windows event logs and API calls, which can support the timely identification of emerging attack strategies [7–11].

For file recovery, obtaining encryption keys and enabling decryption remain essential, with systems such as PayBreak and Cryptographic Service Provider providing relevant tools. Furthermore, blockchain-based transaction tracking has been proposed as a means to identify ransomware operators and analyze their financial transactions [12].

Recent research emphasizes the need to improve existing defense strategies. Automated ML-based approaches, such as Federated Learning, enable organizations to develop detection models without directly sharing sensitive data. AI-driven generation of decoy files and realistic simulation of enterprise environments contribute to the development of more effective decoy-based detection mechanisms. The evaluation of hybrid cryptographic systems has also been suggested as a promising research direction, potentially enabling decryption without full knowledge of the attackers' methods. AI-based behavioral analysis of online transactions likewise offers opportunities to enhance monitoring and detection capabilities.

2.2 Comparative analysis of selected recent studies

A review of recent work highlights a wide variety of detection methods. The Early Ransomware Detection (2023) study [13] combined API call and file system monitoring to address zero-day attacks using behavioral machine-learning models. The ProtectNIC (2023) study [14] integrated network traffic analysis with SmartNIC technology to support real-time detection with low latency. The Applying Staged Event-Driven Access Control (2023) study [15] proposed an automated policy-based system to minimize ransomware activity, albeit at a higher resource cost. The Analysis of Decoy Strategies (2023) study [16] examined limitations of honeypot and decoy systems and reported that static decoys tend to be less effective, whereas dynamically generated decoys can reduce false positives (FP) by up to 75%. Finally, WannaLaugh (2024) [17] developed a ransomware emulator to enable realistic simulations for training and accelerated model learning (Table 1).

Comparative analysis suggests that, while different studies adopt diverse approaches, they share a common goal of early detection and effective prevention. ML- and behavior-based methods remain promising, while SmartNIC technology can enhance network-level protection. The evolution of decoy systems, particularly dynamic

Table 1 Comparison of selected ransomware detection methods

Study	Main focus	Main limitation	Proposed solution	Effectiveness
Early Ransomware Detection (2023) [13]	API calls, file system monitoring	Zero-day detection challenges	Behavioral ML models	Detection rate above 90%
ProtectNIC (2023) [14]	SmartNIC-based detection	Network traffic analysis is time consuming	SmartNIC + ML integration	Real-time detection
Applying Staged Event-Driven Access Control (2023) [15]	Access-control	High resource demand due to user interaction	Automated policy-based protection	Improved decision hierarchy, but resource-intensive
Analysis of Decoy Strategies (2023) [16]	Honeypot and decoy systems	Static decoy files are ineffective	Dynamic decoy files	Up to 75% reduction in false positives (FP)
WannaLaugh (2024) [17]	Ransomware emulator	Lack of real testing environment	Realistic simulations	Faster training of models

honeypots, also contributes to reducing false alarms. Future research may benefit from integrating automated ML techniques, SmartNIC-based defenses, and hybrid decoy strategies.

2.3 AI-based methods for ransomware detection

Machine learning (ML) and deep learning (DL) [18–20] are increasingly used in ransomware defense, as these methods can support the identification of unknown and zero-day threats that evade traditional antivirus systems [5]. In dynamic detection, ML algorithms search for malicious patterns [21], for example, in network traffic [22, 23], where classifiers such as NetConverse examine communication behavior. Similarly, monitoring API calls represents an important component, as models such as LSTM and SVM can be used to identify suspicious encryption-related activity. Windows event log analysis has also been proposed for early detection, with approaches such as DeepRanBiLSTM demonstrating the ability to detect anomalous behavior.

Static analysis offers additional approaches for ransomware detection. ML models [24] can be applied to classify binary structures, such as opcode or byte patterns, using techniques including Random Forest or Gradient Boosting. Portable Executable (PE) header analysis, performed with deep learning models such as convolutional neural networks (CNNs), can support the detection of previously unseen ransomware families. Entropy-based analysis of file activity, combined with ML, represents another complementary technique for identifying the onset of encryption processes [25].

To further enhance detection accuracy, hybrid methods combine static and dynamic analysis [26]. The E2E-RDS (End-to-End Ransomware Detection System) integrates these approaches to support improved detection reliability. Similarly, access-control solutions combined with ML-based monitoring can support the identification of

unauthorized file operations [27]. The use of decoy files and honeypots also plays an important role, as ML-based phishing traps can divert attackers and help reduce the risk of encryption of genuine data.

ML- and DL-based methods thus contribute to improving defenses against rapidly evolving ransomware. These AI-based approaches can support faster detection and enhance resilience to emerging threats, including zero-day attacks. Adaptive models, particularly those analyzing network traffic and API activity, may further strengthen protection. Continuous refinement and updating of such models remain essential to maintain effectiveness over time.

2.4 Other relevant approaches

Recent studies have also explored ransomware detection from complementary perspectives, including kernel-level and I/O-level monitoring, network traffic analysis, and hypervisor-level behavior analysis. These works demonstrate the diversity of existing methodologies and highlight the importance of clearly positioning new contributions.

Peeler et al. [28] proposed a lightweight system with rapid reaction times and low resource consumption, characteristics that align with aspects of our approach. Minerva's resilience against anti-detection techniques [29] similarly informs our focus on robustness. Herrera-Silva et al. [30] introduced a dynamic feature set for ML-based detection of continuously evolving ransomware patterns, such as file I/O and process launches, which is conceptually aligned with our log-based methodology. Lee et al. [31] investigated detection under format-preserving encryption (FPE) scenarios, presenting an FPE-resilient approach that complements our emphasis on server-side log features. Zhou et al. [32] presented the ANIMAGUS system, which evaluates the robustness of I/O-based detection under adversarial conditions, further motivating the need for resilient

system designs. Zahoora et al. [33] developed a cost-sensitive Pareto-ensemble deep learning model for detecting zero-day ransomware, illustrating the potential of ensemble architectures, while our work reports competitive performance using lightweight classical models suitable for practical deployment. Finally, Baldwin and Dehghantanha [34] explored static opcode density analysis with SVMs, demonstrating the continued relevance of static analysis approaches.

Several benchmark studies have shown that modern AES implementations achieve very high throughput across platforms, indicating that raw encryption speed is not a reliable discriminator for ransomware activity. For instance, AES-GCM optimized with AES-NI reaches approximately 1.5–2.5 cycles per byte on contemporary CPUs, as reported by Kampanakis et al. [35] and Gueron et al. [36]. Multi-core processors can sustain multi-gigabit performance (approximately 7 Gbps), based on the results of Barnes et al. [37], and even low-end microcontrollers achieve performance in the hundreds of cycles per byte [38]. GPU-accelerated AES-CTR implementations further extend throughput into the multi-gigabit range [39]. Collectively, these results indicate that encryption itself is sufficiently fast across diverse environments; therefore, encryption speed alone is unlikely to serve as a reliable distinguishing characteristic for ransomware detection. Recent work has also explored behavior-based ransomware detection on mobile platforms [40], memory-centric models for dynamic ransomware analysis [41], optimized deep learning for static malware classification [42], and domain-specific reviews highlighting gaps in current detection techniques, particularly in critical sectors such as healthcare [43].

This extended review highlights the diversity of methodological approaches in the field and helps clarify how our contribution fits within this landscape. While

Peeler emphasizes lightweight and efficient design, Minerva focuses on robustness against evasion techniques. ANIMAGUS addresses the challenges of adversarial resilience, whereas Pareto-ensemble learning illustrates the potential of deep ensemble methods. Against this background, our proposed system can be positioned as a server-side, log-based ML approach that aims to provide efficiency and robustness while supporting quasi real-time detection in practical deployment environments.

3 Methods

3.1 Dataset and simulation environment

For the experimental evaluation, we constructed a virtualized SME environment to approximate realistic operating conditions. The architecture of the evaluated environment and its operational workflows are illustrated in Fig. 1.

The infrastructure comprised a Windows Server instance providing centralized file services and four distinct user profiles generating benign workloads (office editing, bulk file copying, archival/compression, and mixed interactive usage). Five ransomware families (Ryuk, WannaCry, NotPetya, Lockbit, and TeslaCrypt) were executed multiple times in randomized order to avoid bias and to reflect the variability of real-world attack scenarios. During each run, benign user activity was performed in parallel, resulting in a dataset that combined normal and malicious file system operations.

During data collection, server-side logging was configured to record file operation events with nanosecond-resolution timestamps. End-to-end logging throughput (from event emission to durable storage) was measured and remained well below the theoretical limits of the underlying hardware and database engine, indicating that logging did not constitute a performance bottleneck and that events were captured reliably even under elevated load.

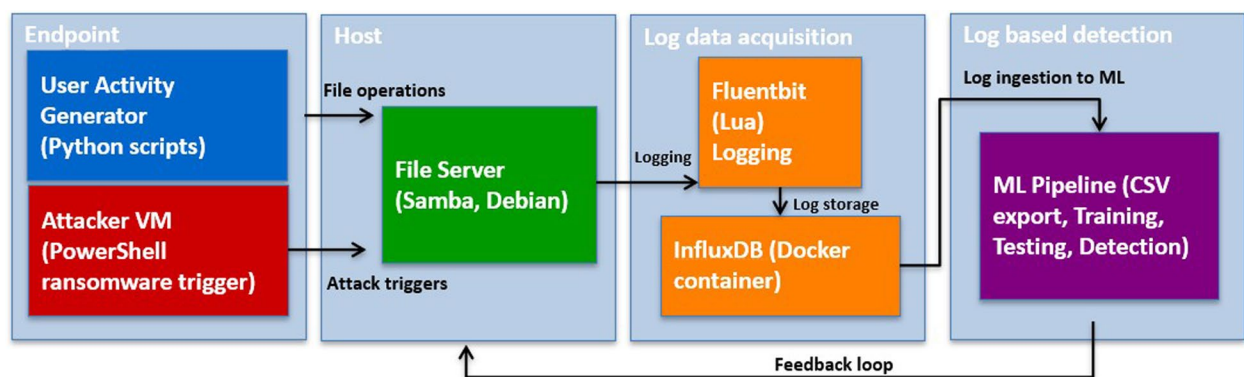


Fig. 1 The examined environment's model and its working flows

3.2 Logging and preprocessing

File operations were logged on the server side, capturing create, write, read, rename, and delete actions. Events were exported via a Unix socket and forwarded by Fluent Bit to an InfluxDB time-series database. SSD-backed storage and a RAM drive were used to minimize ingest latency. For downstream machine learning (ML) pipelines, logs were periodically exported to CSV files.

Raw events were aggregated into non-overlapping one-second windows, producing feature vectors that describe the total number of each file operation type within a window. This aggregation helps smooth high-frequency noise while preserving temporal dynamics characteristic of ransomware activity. To mitigate potential *temporal leakage*, absolute timestamps were not used directly as features. Instead, only relative event counts (and window indices for sequence modeling) were considered, reducing the risk that models rely on dataset-specific time information.

Further dataset refinement included two filtering steps. First, consecutive all-zero windows (no activity observed) were collapsed into a single representative record to reduce redundancy; this representation is referred to as the “one-zero” variant. Second, rows with an active attack label but zero counts (e.g., “0,0,0,1”) were removed, as these reflect preparatory phases without observable file operations and could otherwise introduce misleading patterns. Multiple dataset variants were thus generated, summarized in Table 2.

To mitigate potential temporal leakage, absolute timestamps were not used directly as features. Instead, only relative event counts were included, reducing the risk that the models rely on dataset-specific time information. As a result, the final dataset captures behavioral patterns that are more likely to generalize across different environments and data collection periods.

3.3 Feature engineering

From the aggregated logs, we derived feature groups to capture behavioral aspects of ransomware activity. Core

features included the per-window counts of create, write, read, rename, and delete, which reflect the characteristic read–encrypt–delete sequence associated with ransomware behavior.

Each one-second interval was represented as a single feature vector, allowing machine learning algorithms to operate on uniform, tabular input without requiring sequence-aware models. This design preserved the temporal granularity necessary for near real-time detection while maintaining compatibility with classical and ensemble machine learning classifiers. The binary label `att` (0/1) indicated benign versus attack windows and served as the supervised learning target.

3.4 Machine learning models and hyperparameter tuning

Five classical and ensemble machine learning models were selected for evaluation: Random Forest (RF), Support Vector Machine (SVM), Decision Tree (DT), AdaBoost (ADA), and XGBoost (XGB). All models were trained using stratified sampling to maintain the ratio of benign and malicious samples across both the training and test sets. Hyperparameter optimization was performed using GridSearchCV with k-fold cross-validation. This automated search aimed to identify suitable model configurations by splitting the dataset into k subsets and training the model k times, each time using a different subset as the validation set. This procedure provides a robust estimate of model performance, reducing the risk of overfitting, although it increases computational cost.

The optimization objective prioritized sensitivity (true positive rate), while avoiding excessive degradation of precision, reflecting the practical need to detect attacks early while limiting false alarms.

3.5 Evaluation metrics

To assess detection performance, standard classification metrics were employed. Accuracy, F1-score, sensitivity (true positive rate), and specificity (true negative rate) were computed for each model. Latency was measured as the *prediction time*, reflecting the computational overhead required for a model to classify a given sample and supporting the evaluation of feasibility for quasi real-time operation.

Confusion matrices were generated to provide a detailed breakdown of prediction outcomes (TP, FP, TN, FN). These metrics were selected to provide a comprehensive view of detection effectiveness, capturing not only overall accuracy but also the relationship between sensitivity and specificity, as well as the balance of false positives (FP) and false negatives (FN), which are critical in the context of ransomware detection. In this study, recall and sensitivity are used interchangeably and

Table 2 Dataset representations used in the experiments

Dataset representation	Size (bytes)	Records
result_long_train_ok.csv	63,936,253	63,936,251
result_long_test_csv	31,968,047	31,968,045
result_long_train_ok_nozeroatt.csv	63,581,830	1,718,421
result_long_test_nozeroatt.csv	31,772,835	858,712
result_long_train_ok_onezeroatt.csv	356,230	9,621
result_long_test_onezeroatt.csv	29,425	794

both correspond to the true positive rate (TPR), defined as $TPR = \frac{TP}{TP+FN}$.

In the confusion matrix, true positives (TP) denote correctly identified attack instances, false positives (FP) represent benign activities incorrectly classified as attacks, true negatives (TN) correspond to correctly classified benign events, and false negatives (FN) indicate missed attack instances. Based on these quantities, sensitivity is expressed as the true positive rate (TPR), while specificity corresponds to the true negative rate (TNR).

3.6 ROC point and precision–recall point analysis

Beyond the standard metrics, ROC and precision–recall (PR) point analyses were carried out to visualize classifier performance as single operating points. As this is a binary classification task, all models were evaluated using a fixed decision threshold of 0.5 to ensure consistent and comparable results across classifiers. Consequently, the evaluation yielded a single operating point for each model rather than a full curve. In the ROC space, the true positive rate (TPR) was plotted against the false positive rate (FPR), while in the precision–recall space, precision was plotted against recall. These complementary perspectives illustrate the relationships between sensitivity and specificity, as well as between precision and recall, thereby showing how each model balances early detection against the risk of false alarms and the completeness of detection.

3.7 Statistical validation

The validation of the experimental results was supported by the design of the data collection process. Five distinct ransomware families (Ryuk, WannaCry, NotPetya, Lockbit, and Teslacrypt) were executed in a randomized manner over a period of 30 days. For each run, both the selected ransomware family and the execution duration (ranging from 1 to 6 h) were determined randomly. This procedure was intended to provide unbiased sampling across attack types and time intervals.

The primary objective of this experimental strategy was not to compare ransomware families individually, but to ensure sufficient data coverage for training and evaluation. In realistic scenarios, a single ransomware instance would typically execute once, potentially destroying user files. Accordingly, the experimental setup was designed to generate repeated attack traces under varying conditions to build statistically meaningful datasets. The results indicated that, irrespective of which ransomware family was executed or how frequently runs occurred, the machine learning models exhibited similar proportions of true and false predictions, as reflected in the confusion matrices.

3.8 Cost-sensitive evaluation

Finally, a cost-sensitive analysis was introduced to account for the unequal impact of false positives (FP) and false negatives (FN). Based on the confusion matrix results, a cost matrix was defined with $C_{FP} = 1$ and $C_{FN} = 10$, while $C_{TP} = C_{TN} = 0$. This reflects the practical reality that an undetected ransomware attack (FN) is considerably more costly than a false alarm (FP). The total cost for each model was calculated, and a comparative evaluation was conducted. This approach provided an additional decision criterion that complements standard metrics by prioritizing models that minimize overall operational risk.

4 Development and comparison of machine learning models

4.1 Selected machine learning models

Building on prior experience and the requirements of quasi real-time detection, we focused our evaluation on classical and ensemble machine learning models. Deep learning methods were deliberately excluded, as the dataset consisted of compact feature vectors representing aggregated file operations, which do not warrant high-dimensional architectures. The selected algorithms are well suited for binary classification tasks of moderate complexity, where both computational efficiency and predictive reliability are essential.

Each one-second interval was represented by three aggregated integer values per record, corresponding to read, update, and delete file operation counts. Given this low-dimensional representation, we prioritized models known for robustness and efficiency over deep learning approaches that typically require higher-dimensional inputs.

Classical models, such as Support vector machines (SVM) and Decision Trees (DT), are computationally efficient, handle small datasets effectively, and require comparatively few hyperparameters. Ensemble methods, including Random Forests (RF), Adaptive Boosting (ADA), and Extreme Gradient Boosting (XGB), often achieve improved performance by aggregating predictions from multiple weak learners. This selection covers both interpretable classical classifiers (SVM, DT) and ensemble methods (RF, ADA, XGB) that are known to enhance robustness and generalization. In particular, tree-based ensembles (RF, XGB) are effective at handling heterogeneous feature distributions, while boosting approaches (ADA, XGB) help reduce variance and bias.

These considerations motivated the inclusion of RF, SVM, DT, ADA, and XGB classifiers for ransomware detection based on server-side logs. Hyperparameter tuning aimed to maximize the true positive detection rate while avoiding excessive increases in false positives (FP), thereby improving sensitivity without degrading overall precision.

4.2 Construction of the testing environment, logging methods

A simulation environment was established to emulate the IT infrastructure of a medium-sized SME (10–50 employees) using the Proxmox VE virtualization platform. The infrastructure consisted of three virtual machines connected through a segmented network. A Samba 4.19.3 file server running on Debian 12.4.0 Bookworm provided access to centrally stored files for two Windows 10 Professional clients via mapped network drives. Logging in the required format was implemented via a Unix socket, with Fluent Bit (Lua module) forwarding events to an InfluxDB time-series database running in a Docker container.

In the database, the following attributes were stored for each record (values aggregated per second):

- timestamp (UTC RFC3339)
- nc (integer, aggregated number of file creation operations)
- nr (integer, aggregated number of file renaming operations)
- nu (integer, aggregated number of file unlinking/deletion operations)
- att (binary indicator, taking the value “1” during attack periods, “0” otherwise)

A Windows workstation simulated both on-site and remote user activities according to four predefined file usage patterns (Table 3). These patterns were implemented using Python scripts that continuously scheduled and executed file creation, renaming, and deletion operations.

For the experimental evaluation, ransomware samples were executed in randomized order, generating log data with varying attack durations. In total, attacks from five distinct ransomware families (Ryuk, NotPetya, Lockbit, Teslacrypt, and WannaCry) were performed, with

multiple runs per family. The randomized execution order was intended to support unbiased evaluation, while the distribution of data across families was balanced with respect to log length and diversity, reducing the risk of dominance by any single family.

The Windows 10 host designated as the attacker executed ransomware samples via a PowerShell script, with a random delay of 1 to 6 h before launching one of the five ransomware families. Concurrently, the `att` field in the InfluxDB “attack” bucket (running in a container on the Proxmox VE server) was set to “1” and timestamped via the HTTP API. Following each attack, six representative file types (docx, pdf, ppt, pptx, xlsx, zip), with a combined size of 45,534,809 bytes, were recovered. As the size of user-generated docx files was negligible (30,162 bytes per file), this contributed to rapid completion of the encryption process.

A Bash monitoring script running on the Proxmox VE server periodically (every second) checked the existence of a canary file (`b.docx`) via the NFS protocol. This file was excluded from rename and delete operations in the user scripts and could therefore only be altered by active ransomware. If the file was not found, the monitoring script inserted “0” into the `att` field, restored the attacker host to a clean snapshot, deleted all modified content on the shared drive, and restored the original files. Consequently, transitions between values of 0 and 1 in the `att` field reliably delineated attack periods, while the remaining intervals indicated benign system activity.

4.3 Data cleansing and preprocessing

The raw datasets contained large numbers of consecutive all-zero records, corresponding to idle periods without file operations. If retained, these records would have dominated the training process and biased the models toward the benign class while adding little

Table 3 Job templates and associated file operation activity ranges

Job template	Working hours	Creating files (count)	Renaming files (count)	Deleting files (count)
Users at work	08:00–16:00	30–60	10–30	5–20
Users at remote work	16:00–08:00	20–60	10–20	5–10
Secretary	08:00–16:00	30–60	10–20	5–20
Secretary at remote work	16:00–08:00	20–30	5–10	5–10
IT admin at work	08:00–16:00	30–50	10–30	10–20
IT admin at remote work	16:00–08:00	0–20	10–20	0–20
CEO at work	08:00–16:00	0–10	0–5	0–5
CEO at remote work	16:00–08:00	0–5	0–2	0–2

informational value. To address this issue, at most one record from each sequence of consecutive all-zero entries was retained, thereby compressing uneventful periods into a single representation. This dataset variant is referred to as the “one-zero” representation (`train_onezero.csv`, `test_onezero.csv`).

Additionally, rows labeled as attacks but containing only zero-valued file operation counts (e.g., “0,0,0,1”) were excluded. Such rows represent preparatory phases of ransomware execution (e.g., process initialization, host infection, or command-and-control activity) without observable file operations. Since the focus of this study is on file I/O behavior, these entries were not suitable for training and could have introduced noise. The resulting dataset variants therefore helped ensure that training concentrated on file-operation-based patterns of ransomware activity. Table 4 illustrates a sample of the processed dataset.

The collected log dataset was divided into training, testing, and validation subsets using an 80–10–10 split for model development and evaluation.

4.4 Model training, hyperparameter tuning, and evaluation

All models were first trained using default hyperparameters to establish baseline performance. Hyperparameter tuning was subsequently performed using `GridSearchCV` with default 5-fold cross-validation. The optimization objective prioritized sensitivity (true positive rate, TPR), reflecting the higher cost of false negatives (FN) in ransomware detection. Hyperparameter search spaces were defined based on exploratory tuning runs to avoid extreme or degenerate values.

Performance was evaluated using Accuracy, F1-score, Sensitivity (Recall), Specificity, and confusion matrices. Final model configurations were selected based on their ability to improve sensitivity while avoiding excessive degradation of precision. The resulting tuned parameters are reported in the Sect. 5 (Table 6).

4.4.1 Evaluation metrics

For each model, the following measures were computed, and model performance was evaluated using the following indicators:

- **Accuracy:** proportion of correctly classified samples.
- **F1-score:** harmonic mean of precision and recall, balancing false positives (FP) and false negatives (FN).
- **Specificity (true negative rate):**

Table 4 Sample of the “one-zero” dataset (“`test_onezero_nozeroatt.csv`”)

Index	Timestamp	nc	nr	nu	att
169	2024-06-07 05:46:24+00:00	2	0	0	0
170	2024-06-07 05:46:25+00:00	0	0	0	0
171	2024-06-07 06:00:10+00:00	0	0	2	0
172	2024-06-07 06:00:11+00:00	0	0	0	0
173	2024-06-07 06:02:01+00:00	0	0	1	0
174	2024-06-07 06:02:02+00:00	0	0	0	0
175	2024-06-07 06:02:31+00:00	1	0	0	0
176	2024-06-07 06:02:32+00:00	0	0	0	0
177	2024-06-07 06:10:50+00:00	5	2	1	1
178	2024-06-07 06:10:59+00:00	1	2	0	1
179	2024-06-07 06:11:00+00:00	19	17	0	1
180	2024-06-07 06:11:46+00:00	0	0	21	1
181	2024-06-07 06:11:59+00:00	0	0	0	0
182	2024-06-07 06:15:07+00:00	0	1	0	0
183	2024-06-07 06:15:08+00:00	0	0	0	0
184	2024-06-07 06:20:09+00:00	0	0	1	0
185	2024-06-07 06:20:10+00:00	0	0	0	0

$$\frac{TN}{TN + FP}$$

- **Sensitivity (true positive rate/recall):**

$$\frac{TP}{TP + FN}$$

Together, these metrics provide a comprehensive view of classification performance. Sensitivity is particularly critical in this context, as an undetected ransomware attack (FN) can lead to severe consequences. Specificity and negative predictive value (NPV) complement this perspective by characterizing reliability on benign traffic, while the F1-score balances precision and recall. Collectively, these indicators capture both the ability to detect attacks and the reliability of predictions under varying outcome distributions.

4.4.2 Complementary analyzes

To complement the standard performance metrics, we also considered two additional evaluation perspectives:

- **ROC point and precision–recall point analysis:** since all models were evaluated as binary classifiers using a fixed decision threshold of 0.5, each model appears as a single operating point in ROC and precision–recall spaces. These points illustrate the relationships between sensitivity, specificity, precision, and recall.

- **Cost-sensitive evaluation (conceptual schema):** a cost matrix was introduced to account for the asymmetric impact of misclassifications, assigning higher weight to false negatives (FN) than to false positives (FP), while true positives (TP) and true negatives (TN) incur no cost. This schema reflects the practical consequences of undetected ransomware activity and defines the evaluation framework. The cost schema is presented in the Section 5 (Table 8).

Together, these evaluation strategies ensure that models are compared not only by raw predictive performance, but also with respect to their practical suitability for ransomware detection based on server-side file operation logs. Detailed numerical results and comparisons are discussed in Section 5.

Although full ROC curves and log-loss values were not computed across repeated runs, hyperparameter tuning with `GridSearchCV` ensured that the selected models represent well-performing configurations within their respective parameter spaces. To complement the standard metrics, the following additional analyses were conducted:

- **ROC point and precision–recall point analysis:** each model is represented by a single operating point in the respective space, derived from true positive rate (TPR), false positive rate (FPR), precision, and recall values obtained from the confusion matrix.
- **Cost-sensitive evaluation (model-specific results):** Using the defined cost schema, total costs were computed for each trained model based on observed confusion-matrix outcomes. While TP and TN incur zero cost, FP contributes a moderate cost due to false alarms, and FN incurs a high cost due to undetected ransomware activity. The resulting model-specific cost values are reported in Table 5.

This comprehensive framework ensured that models were evaluated not only using standard performance metrics, but also in the context of practical deployment considerations, where the relative impact of misclassifications is inherently asymmetric. By integrating classical metrics, statistical indicators, and cost-sensitive evaluation, we established a robust basis for comparing classifiers under realistic operating conditions.

5 Results

5.1 Logging performance and dataset preparation

During data collection, server-side logging performance did not present a measurable bottleneck. File operation events were recorded with nanosecond-resolution

Table 5 Illustrative cost matrix associated with confusion matrix outcomes

	Predicted positive	Predicted negative
Actual positive	TP: 0 (correct detection)	FN: C_{FN} (high cost, undetected attack)
Actual negative	FP: C_{FP} (moderate cost, false alarm)	TN: 0 (correct rejection)

timestamps, and storage throughput remained well below the theoretical limits of the underlying database engine. This indicates that the generated datasets are complete and reliable. For subsequent analysis, log entries were exported to CSV format, cleaned, and processed into multiple dataset variants, as summarized in Table 2.

5.2 Hyperparameter tuning

The machine learning models were first evaluated using default hyperparameters to establish baseline performance, after which exhaustive tuning was performed using `GridSearchCV`. The hyperparameter search spaces and final configurations are reported in Table 6. The optimization objective was to maximize sensitivity (true positive rate, TPR) while maintaining an appropriate balance with precision.

5.3 Model performance

The chart in Fig. 2 represents attack states (0 = no attack, 1 = attack active) over time, providing a clear visualization of the onset and duration of each attack. In Fig. 3, each bar corresponds to one model, with height indicating the prediction latency in seconds. This view complements the attack-state timeline by quantifying recognition latency and facilitating comparisons across models.

Table 7 summarizes the pre- and post-tuning classification performance of all evaluated models, including confusion matrix statistics. Across all models, sensitivity exceeded 84% after tuning, with XGBoost achieving the highest detection rate of 91.9%. This corresponds to detecting 192 out of 209 attack seconds in the test dataset. Prediction latencies remained within the millisecond range (Fig. 3), indicating feasibility for quasi real-time deployment.

Figure 3 shows the latency comparison across the evaluated classifiers. The measured model prediction times range between 0.00019 s and 0.01208 s across all evaluated models, corresponding to millisecond-scale prediction latency. This magnitude supports the feasibility of near real-time operation and approximates real-time behavior under time-critical conditions.

Figure 4 and Table 7 further illustrate performance differences among the tuned models, while Fig. 5 highlights

Table 6 Hyperparameter tuning spaces and the best parameter sets

	Hyperparameter tuning space	Best hyperparameter set
RF	n_estimators: [10, 25, 50, 100, 150] Criterion: [gini, entropy] Max_depth: range(2,6) Min_sample_split: range(2,6) Min_samples_leaf: range(1,6)	Criterion: entropy Max_depth: 5 Min_samples_leaf: 1 Min_sample_split: 2 n_estimators: 25
SVM	Criterion: [gini, entropy] Max_depth: range(2,10) Min_sample_split: range(2,10) Min_samples_leaf: range(1,10)	Estimator_criterion: gini Estimator_max_depth: 4 Estimator_min_samples_leaf: 2 Estimator_min_samples_split: 4 Learning_rate: 1 n_estimators: 100
DT	Criterion: [gini, entropy] Max_depth: range(2,10) Min_sample_split: range(2,10) Min_samples_leaf: range(1,10)	Criterion: entropy Max_depth: 9 Min_samples_leaf: 2 Min_samples_split: 9
ADA	Estimator_criterion: [gini, entropy] n_estimators: [50, 100, 150, 200] Learning_rate: [0.1, 1, 10] Estimator_max_depth: range(2,5) Estimator_min_sample_split: range(2,5) Estimator_min_samples_leaf: range(1,5)	Estimator_criterion: gini Estimator_max_depth: 4 Estimator_min_samples_leaf: 2 Estimator_min_samples_split: 4 Learning_rate: 1 n_estimators: 100
XGB	n_estimators: [50, 100, 150, 200] Learning_rate: [0.2, 0.3, 0.5, 1] Subsample: [0.3, 0.4, 0.5, 0.7, 1] Gamma: [0, 1, 3, 5] Max_depth: [4, 5, 6, 10] Tree method: [hist] Grow policy: [lossguide] Min_child_weight: [0, 3, 5] Max_delta_step: [0, 1, 2, 5, 10] Lambda: [0, 1, 2] Alpha: [0, 1, 2]	Alpha: 0 Gamma: 5 Grow_policy: lossguide Lambda: 0 Learning_rate: 1 Max_delta_step: 0 Max_depth: 5 Min_child_weight: 0 n_estimators: 50 Subsample: 1 Tree_method: hist

false prediction counts. XGBoost and Decision Tree demonstrated consistently strong performance, with XGBoost achieving the best overall results among the evaluated classifiers.

5.4 ROC point and precision–recall point analyses

To complement the above metrics, ROC point and precision–recall point analyses were performed (Figs. 6, 7, 8 and 9). Parameter tuning generally improved model placement in ROC and PR spaces, with XGBoost exhibiting the most favorable balance between recall, precision, and false positive rate (FPR). In contrast, SVM showed more limited gains and, in some cases, a reduction in recall.

5.5 Cost-sensitive evaluation

To reflect the unequal consequences of misclassifications, a cost matrix analysis was performed (Table 8). With $C_{FP} = 1$ and $C_{FN} = 10$, XGBoost yielded the lowest total cost among the evaluated models, indicating a favorable balance between detection capability and reliability. By contrast, SVM incurred the highest cost due to a larger number of false negatives (FN), suggesting more limited suitability for ransomware detection in this setting.

5.6 Summary of results

The results indicate that ransomware activity produces detectable behavioral traces in server-side file operation logs. Classical models, such as Decision Tree, achieved competitive performance; however, ensemble methods,

Table 7 Classification performance of the evaluated models before and after hyperparameter tuning, including confusion matrix statistics (TP, FP, TN, FN)

		RF	SVM	DT	ADA	XGB
Before tuning	Time for prediction (sec)	0.00208	0.00341	0.00019	0.01208	0.00093
	Accuracy	0.90306	0.73724	0.87755	0.75000	0.94133
	F1 score	0.90256	0.68111	0.87879	0.75000	0.94236
	Sensitivity (TPR)	0.84211	0.52632	0.83254	0.70335	0.89952
	Specificity (TNR)	0.97268	0.97814	0.92896	0.80328	0.98907
	Confusion matrix	TP:176	TP:110	TP:174	TP:147	TP:188
		FP:5	FP:4	FP:13	FP:36	FP:2
		TN:178	TN:179	TN:170	TN:147	TN:181
		FN:33	FN:99	FN:35	FN:62	FN:21
Tuned models	Accuracy	0.85714	0.85714	0.94133	0.92602	0.94288
	F1 score	0.86408	0.86275	0.94236	0.92621	0.94581
	Sensitivity (TPR)	0.85167	0.84211	0.89952	0.87081	0.91866
	Specificity (TNR)	0.86339	0.87432	0.98907	0.98907	0.97268
	Confusion matrix	TP:178	TP:176	TP:188	TP:182	TP:192
		FP:25	FP:23	FP:2	FP:2	FP:5
		TN:158	TN:160	TN:181	TN:181	TN:178
		FN:31	FN:33	FN:21	FN:27	FN:17

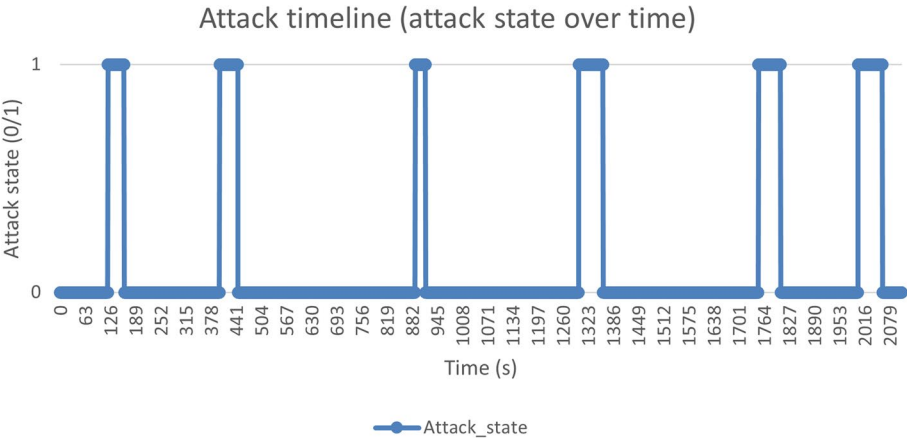


Fig. 2 Attack timeline

and in particular XGBoost, consistently exhibited a more favorable balance between detection accuracy, prediction latency, and cost sensitivity. These findings support the feasibility of quasi real-time ransomware detection using lightweight machine learning models deployed at the server side.

In total, more than 2.5 million log entries were collected over a 30-day period. After data cleansing, the resulting datasets were used for training, testing, and validation of the machine learning models. Beyond supporting the evaluation framework, analysis of the raw logs also informed the development of the logging

methodology itself. By introducing refined data representations, dataset size was substantially reduced while preserving relevant behavioral information, thereby improving processing efficiency and scalability.

6 Discussion and conclusions
6.1 Discussion

The proposed log-based detection framework shows strong potential for integration into existing security infrastructures, such as Security Information and Event Management (SIEM) platforms and Endpoint Detection and Response (EDR) systems. Its lightweight design

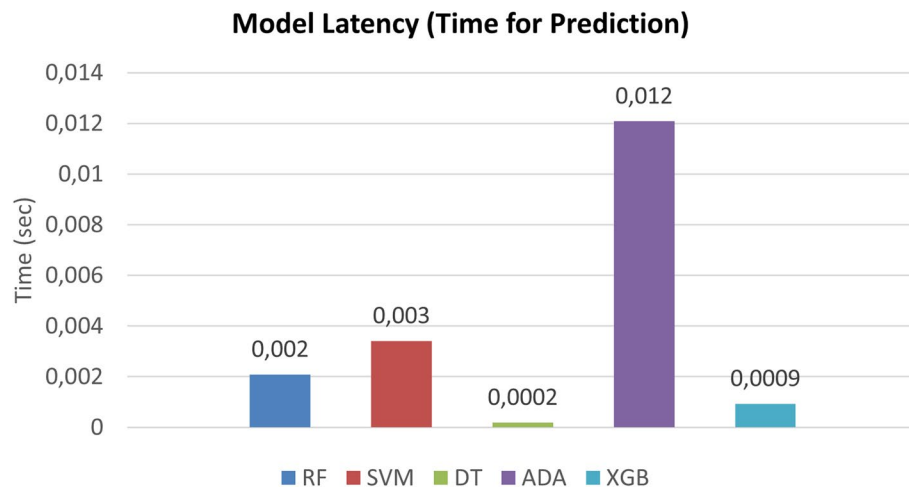


Fig. 3 Prediction latency comparison across models

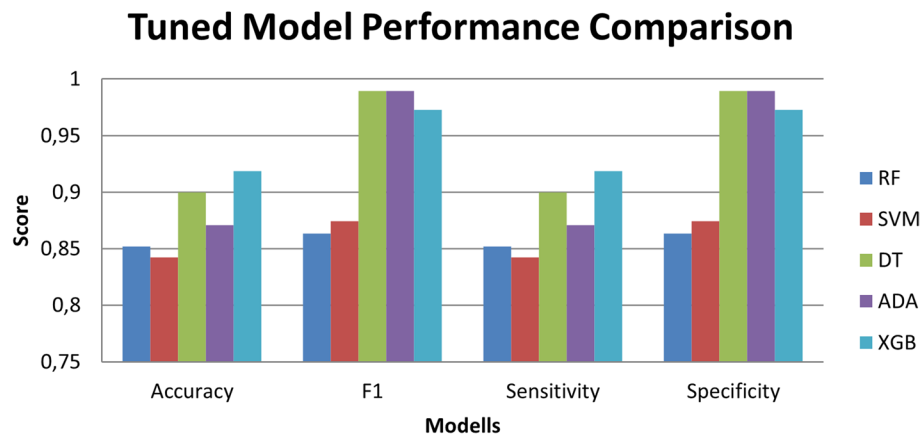


Fig. 4 Performance comparison of tuned models

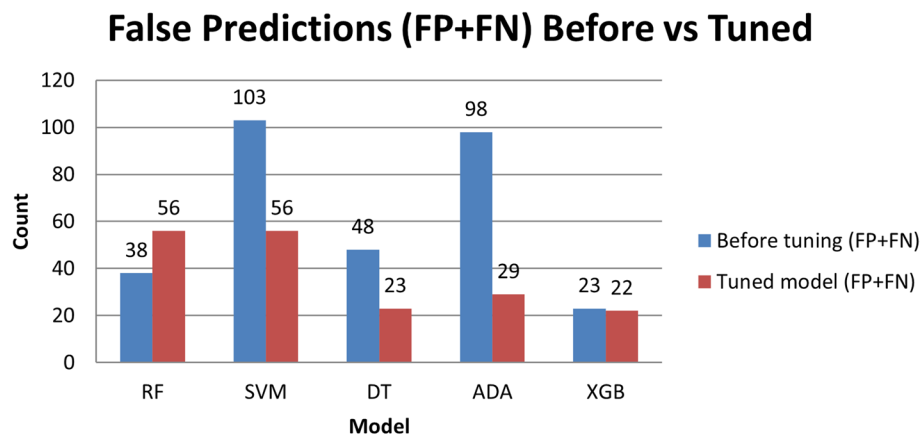


Fig. 5 False predictions across tuned models

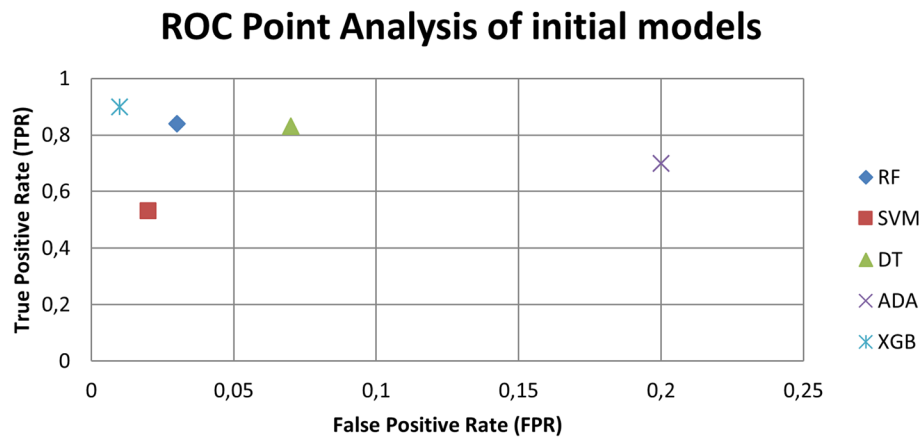


Fig. 6 ROC points of initial models

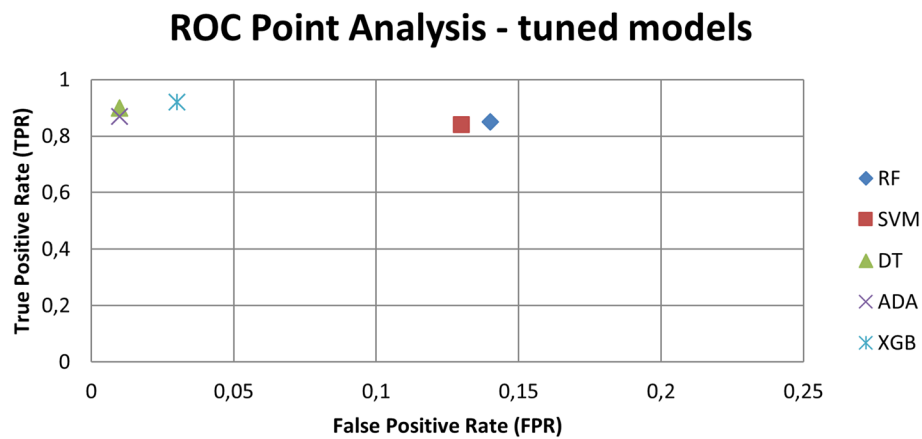


Fig. 7 ROC points of tuned models

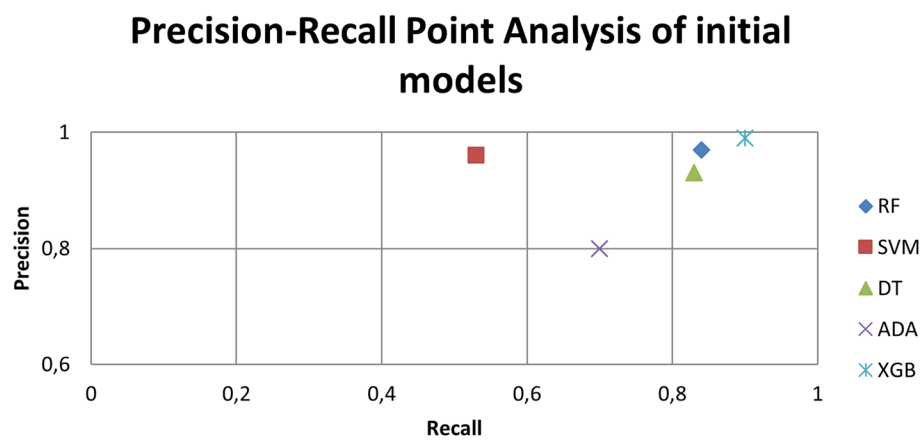


Fig. 8 Precision–recall points of initial models

results in minimal CPU and memory overhead, supporting quasi real-time operation without requiring intrusive kernel- or hypervisor-level modifications. Runtime

efficiency experiments indicated that prediction latency remains within the millisecond range, and the framework exhibits robustness under naturally imbalanced data

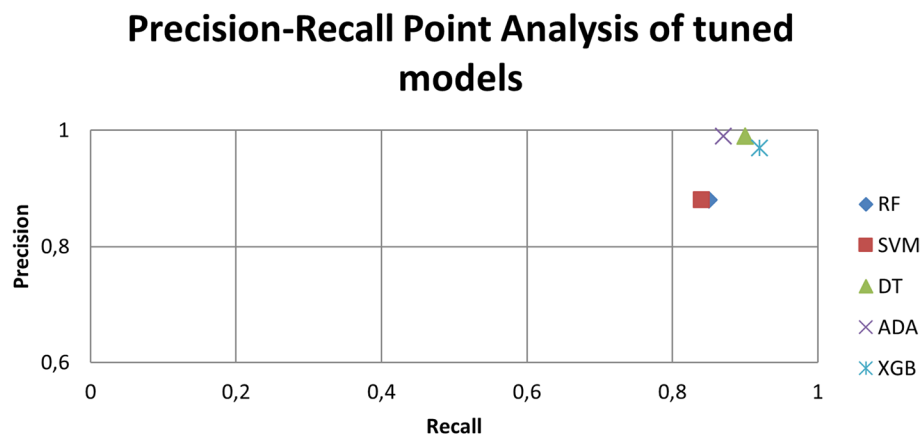


Fig. 9 Precision–recall points of tuned models

Table 8 Cost matrix analysis of the models ($C_{TP} = 0$, $C_{TN} = 0$, $C_{FP} = 1$, $C_{FN} = 10$)

	RF	SVM	DT	ADA	XGB
FP	25	23	2	2	5
FN	310	330	210	270	170
Total cost	335	353	212	272	175

streams, which are representative of realistic deployment environments.

Accordingly, the term quasi real-time is used throughout the manuscript to describe the implemented aggregation-based processing pipeline, while real-time operation is discussed as a feasible approximation supported by millisecond-scale event logging and prediction latency, rather than as a strict hard real-time guarantee.

6.2 Strengths and limitations

The strengths and limitations of the proposed approach are summarized in Table 9. The method is resilient to obfuscation and dormant ransomware variants, as detection relies on behavioral patterns in file operations

rather than on static signatures. The completeness of log collection is also supported, since file operations with timestamps are systematically recorded. The primary limitation is the dependence on a sufficiently representative training dataset, which may affect portability across different organizational environments. Nevertheless, this limitation does not diminish the overall feasibility of the approach, but rather highlights the importance of careful dataset design and continuous data collection strategies.

6.3 Comparison with existing detection paradigms

We compared our results with currently widespread detection paradigms. Signature-based antivirus systems are well known to struggle with consistently detecting zero-day or obfuscated variants [44]. Entropy-based detectors, in turn, are vulnerable to evasion techniques (e.g., Base64 encoding and partial encryption) and typically incur considerable computational overhead [45, 46]. Similar concerns are echoed in recent studies highlighting the relationship between I/O-based detection accuracy and runtime efficiency [47]. Moreover, even advanced ML classifiers applied to high-entropy encrypted streams can impose significant resource demands [48]. In this context, our server-side log-based ML approach

Table 9 Strengths and limitations of the proposed approach

Strengths	Limitations
- Lightweight: minimal CPU and memory overhead, suitable for quasi real-time use	- Dependent on availability of a sufficiently representative training dataset
- Does not require extensive dataset curation to cover diverse ransomware runs	-
- Easy integration into SIEM/EDR systems without intrusive kernel modifications	-
- Naturally robust to imbalanced data streams, reflecting real-world conditions	-
- Insensitive to obfuscation/dormant variants (detection focuses on behavior, not signature)	-
- Log completeness guaranteed (file operations with timestamps are fully captured)	-

Table 10 Key supporting references and their demonstrated insights on ransomware detection paradigms

Source	What it supports/shows
Kothamali and Banik (2025): Limitations of Signature-Based Threat Detection [44]	Signature-based systems struggle to react to new (zero-day) threats and obfuscated malware variants.
McIntosh, Jang-Jaccard, Watters, Susnjak (2023): The Inadequacy of Entropy-Based Ransomware Detection [46]	Entropy-based detectors can be easily evaded (e.g., Base64 encoding, partial encryption); entropy values are manipulable.
Van Sloun et al. (2025) Detecting Ransomware Despite I/O Overhead: A Practical Multi-Staged Approach [47]	I/O-based monitoring is effective but introduces measurable overhead, showing the relationship between accuracy and efficiency.
Kim (2022) Byte Frequency Based Indicators for Crypto-Ransomware [45]	Byte frequency and entropy-based methods introduce overhead for large files; highlights resource cost considerations.
Casino et al. (2025) Not on My Watch: Ransomware Detection through Classification [48]	Examines encrypted bit streams and shows ML approaches can distinguish them but often at significant computational cost.

demonstrates competitive detection performance while maintaining comparatively low computational overhead. Table 10 summarizes these comparative observations.

In relation to the formulated research questions, the findings can be summarized as follows. For the first research question (RQ1), although the experimental setup modeled the behavior of a small SME environment with a single workstation, the measurements indicate that, when using the XGBoost ensemble model, prediction latency remains within the quasi real-time range even when extrapolated to scenarios involving several hundred clients.

For the second research question (RQ2), the applied logging infrastructure (Unix socket, Fluent Bit, InfluxDB, and Lua) and preprocessing steps enabled machine learning models to react to attacks with sufficient speed to support quasi real-time operation.

For the third research question (RQ3), while precise quantification of data loss is not possible due to the use of one-second aggregation intervals and canary-file-based termination, the experimental results indicate that ransomware detection can occur before extensive file damage takes place. Potential improvements include reducing the aggregation window or introducing predictive execution control mechanisms.

6.4 Conclusions

The study indicates that ransomware attacks leave consistent and detectable behavioral traces in server-side file operation logs. By leveraging lightweight classical and ensemble machine learning models, particularly XGBoost, the proposed framework achieves a favorable balance between detection accuracy, prediction latency, and cost sensitivity. The solution represents a scalable and resource-efficient alternative to file server-focused defenses, demonstrating robust performance under naturally imbalanced workloads.

Future work will focus on further validation in large-scale enterprise environments, incorporating more

diverse datasets to improve generalizability, and exploring adaptive retraining mechanisms to maintain resilience against evolving ransomware strategies. Overall, these findings highlight the practical applicability of log-based, server-side machine learning approaches for quasi real-time ransomware detection and their potential to strengthen the defensive capabilities of modern security infrastructures.

Acknowledgements

We would like to thank Ms. Ágnes Vathy-Fogarassy for her professional support related to the methodology of machine learning models and for her valuable suggestions regarding the structure of the article. We are also grateful to Mr. Szilveszter Tóth for his assistance in building the Docker and database infrastructure, as well as for sharing practical insights that contributed to the successful completion of this work.

Authors' contributions

GA contributed to the design of the test environment, developed the methodology, and carried out data collection and preprocessing. VSZ defined the research objectives, contributed to the validation of the results, and coordinated the writing process. GA and VSZ were responsible for drafting the manuscript. TM contributed to the validation of the results. All authors contributed to the interpretation of the results and reviewed and approved the final version of the manuscript.

Funding

Open access funding provided by University of Pannonia.

Data availability

The datasets generated and recorded as part of this study are available from the corresponding author upon reasonable request.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare that they have no competing interests.

Received: 29 May 2025 Accepted: 6 February 2026
Published online: 09 March 2026

References

1. E.U.A. for Cybersecurity (ENISA). Enisa threat landscape 2024 (2024). <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2024>. Accessed 29 May 2025
2. Verizon. Verizon data breach investigations report 2025 (2025). <https://www.verizon.com/business/resources/reports/dbir/>. Accessed 29 May 2025
3. ENISA. Enisa threat landscape: Finance sector 2024 (2025). https://www.enisa.europa.eu/sites/default/files/2025-02/Finance%20TL%202024_Final.pdf. Accessed 29 May 2025
4. Europol. Internet organised crime threat assessment (iocta) 2024 (2024). <https://doi.org/10.2813/442713>
5. Y. Guo, A review of machine learning-based zero-day attack detection: challenges and future directions. *Comput. Commun.* **198**, 175–185 (2023). <https://doi.org/10.1016/j.comcom.2022.11.001>
6. P. Singh, S.K. Borgohain, A.K. Sarkar, J. Kumar, L.D. Sharma, Feed-forward deep neural network (ffdn)-based deep features for static malware detection. *Int. J. Intell. Syst.* **2023**(1), 9544481 (2023). <https://doi.org/10.1155/2023/9544481>
7. R. Kaur, M. Singh, A survey on zero-day polymorphic worm detection techniques. *IEEE COMMUNICATIONS SURVEYS AND TUTORIALS* **16**(3), 1520–1549 (2014). <https://doi.org/10.1109/SURV.2014.022714.00160>
8. R. Ahmad, I. Alsmadi, W. Alhamdani, L. Tawalbeh, Zero-day attack detection: a systematic literature review. *Artif. Intell. Rev.* **56**(10), 10733–10811 (2023). <https://doi.org/10.1007/s10462-023-10437-z>
9. U.K. Singh, C. Joshi, D. Kanellopoulos, A framework for zero-day vulnerabilities detection and prioritization. *JOURNAL OF INFORMATION SECURITY AND APPLICATIONS* **46**, 164–172 (2019). <https://doi.org/10.1016/j.jisa.2019.03.011>
10. J. Sebastian Guisao, J.C. Toro Rendon, Mitigation and detection of zero-day vulnerabilities. *CUADERNO ACTIVA* **6**, 63–67 (2014)
11. U. Tupakula, V. Varadharajan, in *2011 7TH International Wireless Communications and Mobile Computing Conference (IWCMC)*, Security techniques for zero day attacks (IEEE/IEEE Turkey sect, 2011), pp. 442–447
12. A. AlEroud, G. Karabatis, in *2012 ASE International Conference on Cyber Security (CYBERSECURITY)*, A contextual anomaly detection approach to discover zero-day attacks (Acad Sci & Engn; IEEE Comp Soc, 2012), pp. 40–45. <https://doi.org/10.1109/CyberSecurity.2012.12>
13. M. Cen, F. Jiang, X. Qin, Q. Jiang, R. Doss, Ransomware early detection: a survey. *Comput. Netw.* **239**, 110138 (2024). <https://doi.org/10.1016/j.comnet.2023.110138>
14. A. Xu, A. Choudhury, E. Liu, S. Choi, in *2024 Silicon Valley Cybersecurity Conference (SVCC)*, Protectnic: Smartnic-based ransomware detection (2024), pp. 1–3. <https://doi.org/10.1109/SVCC61185.2024.10637308>
15. Applying staged event-driven access control to combat ransomware-Web of Science Core Collection (2025). <https://www.webofscience.com/wos/woscc/full-record/WOS:000953073900001>. Accessed 14 Feb 2025
16. B. Denham, D.R. Thompson, in *2023 IEEE Conference on Communications and Network Security (CNS)*, Analysis of decoy strategies for detecting ransomware (2023), pp. 1–6. <https://doi.org/10.1109/CNS59707.2023.10288691>
17. D. Diamantopoulos, R. Pletka, S. Sarafijanovic, A.L.N. Reddy, H. Pozidis, in *ACM Conferences, WonnaLaugh: A Configurable Ransomware Emulator - Learning to Mimic Malicious Storage Traces* (Association for Computing Machinery, 2024), pp. 118–131. <https://doi.org/10.1145/3688351.3689163>
18. H. Hindy, R. Atkinson, C. Tachtatzis, J.N. Colin, E. Bayne, X. Bellekens, Utilising deep learning techniques for effective zero-day attack detection. *Electronics* (2020). <https://doi.org/10.3390/electronics9101684>
19. F. Deldar, M. Abadi, Deep learning for zero-day malware detection and classification: a survey. *ACM Comput. Surv.* (2024). <https://doi.org/10.1145/3605775>
20. S. Millar, N. McLaughlin, J.M. del Rincon, P. Miller, Multi-view deep learning for zero-day android malware detection. *J. Inf. Secur. Appl.* (2021). <https://doi.org/10.1016/j.jisa.2020.102718>
21. K. Ganame, M.A. Allaire, G. Zagdene, O. Boudar, in *Intelligent, Secure, and Dependable Systems in Distributed and Cloud Environments (ISDDC 2017)*, *Lecture Notes in Computer Science*, vol. 10618, ed. by I. Traore, I. Woungang, A. Awad, Network behavioral analysis for zero-day malware detection - a case study (2017), pp. 169–181. https://doi.org/10.1007/978-3-319-69155-8_13
22. F. Wangde, I. Mulay S.P., R.B. Adhao, V.K. Pachghare, Zero-day attack detection using ensemble technique. *Int. J. Next-Gener. Comput* **12**(5, SI), 551–557 (2021)
23. H. Al-Rushdan, M. Shurman, S.H. Alnabelsi, Q. Althebyan, in *2019 International ARAB Conference on Information Technology (ACIT)*, Zero-day attack detection and prevention in software-defined networks (Al Ain Univ, 2019), International Arab Conference on Information Technology ACIT, pp. 278–282. <https://doi.org/10.1109/aci47987.2019.8991124>
24. Z. He, A. Rezaei, H. Homayoun, H. Sayadi, in *Proceedings of the 32nd Great Lakes Symposium on VLSI 2022, GLSVLSI 2022*, Deep neural network and transfer learning for accurate hardware-based zero-day malware detection (Assoc Comp Machinery; ACM Special Interest Grp Design Automat; IEEE Council Elect Design Automat; IEEE, 2022), pp. 27–32. <https://doi.org/10.1145/3526241.3530326>
25. S.J. Bu, S.B. Cho, Deep character-level anomaly detection based on a convolutional autoencoder for zero-day phishing url detection. *Electronics* (2021). <https://doi.org/10.3390/electronics10121492>
26. F. Manzoor, V. Khattar, C.C. Liu, M. Jin, in *2024 IEEE International Conference on Communications, Control, and Computing Technologies for Smart GRIDS, SMARTGRIDCOMM 2024*, Zero-day attack detection in digital substations using in-context learning (The Research Council of Norway, 2024), International Conference on Smart Grid Communications, pp. 220–225. <https://doi.org/10.1109/SmartGridComm60555.2024.10738025>
27. C. Kim, S.Y. Chang, J. Kim, D. Lee, J. Kim, Automated, reliable zero-day malware detection based on autoencoding architecture. *IEEE Trans. Netw. Serv. Manag.* **20**(3), 3900–3914 (2023). <https://doi.org/10.1109/TNSM.2023.3251282>
28. M.E. Ahmed, H. Kim, S. Camtepe, S. Nepal, in *ESORICS (Lecture Notes in Computer Science*, vol. 12972), *Peeler: Profiling kernel-level events to detect ransomware* (Springer, 2021), pp. 240–260. https://doi.org/10.1007/978-3-030-88418-5_12
29. D. Hitaj, G. Pagnotta, F. De Gaspari, L. De Carli, L.V. Mancini, Minerva: A file-based ransomware detector (2023). arXiv preprint [arXiv:2301.11050](https://arxiv.org/abs/2301.11050)
30. S. Herrera-Silva: Dynamic feature dataset for ransomware detection using behavioral patterns. *Sensors* **23**(3) (2023). <https://doi.org/10.3390/s23031053>
31. F. Lee, ..., Machine learning-based ransomware detection method for files infected with fpe. *Sensors* **25**(8), 2406 (2025). <https://doi.org/10.3390/s25082406>
32. C. Zhou, L. Guo, Y. Hou, Z. Ma, Q. Zhang, M. Wang, Z. Liu, Y. Jiang, in *Proceedings of IEEE Symposium on Security and Privacy (S&P)*, Limits of i/o based ransomware detection: An imitation based attack (2023), pp. 2584–2601. <https://doi.org/10.1109/SP46125.2023.10179372>
33. U. Zahoor, A. Aziz, A. Ashraf, M. Rajarajan, A. Khan, Cspe-r: ransomware detection using deep auto-encoders and pareto ensemble. *Sci. Rep.* **12**, 15421 (2022). <https://doi.org/10.1038/s41598-022-19443-7>
34. J. Baldwin, A. Dehghantanha, Efficacy of opcode density via svm for crypto-ransomware detection (2018). arXiv preprint [arXiv:1807.10442](https://arxiv.org/abs/1807.10442)
35. P. Kampanakis, in *NIST Workshop on Block Cipher Modes of Operation*, Practical challenges with aes-gcm: Performance, side channels, and misuse resistance (2024). <https://csrc.nist.gov/csrc/media/Events/2023/third-workshop-on-block-cipher-modes-of-operation/documents/accepted-papers/Practical%20Challenges%20with%20AES-GCM.pdf>. Accessed 29 May 2025
36. S. Gueron, V. Krasnov, in *Proceedings of the 12th International Conference on Cryptology and Network Security (CANS)*, Aes-gcm software performance on intel architecture processors (Springer, 2013), pp. 65–93. https://doi.org/10.1007/978-3-319-02937-5_5
37. A. Barnes, J. Smith, D. Greg, Improving the throughput of the aes algorithm with multicore processors (2014). arXiv preprint [arXiv:1403.7295](https://arxiv.org/abs/1403.7295)
38. K. Kim, H. Lee, J. Lim, Practical aes-gcm encryption for low-end micro-controllers. *Appl. Sci.* **10**(9), 3131 (2020). <https://doi.org/10.3390/app10093131>
39. K. Assa-Agyei, Performance evaluation of gpu versus cpu implementations of aes-ctr. *Int. J. Comput. Appl.* **182**(40), 12–18 (2023). <https://doi.org/10.5120/ijca2023922941>
40. G. Kirubavathi, W.R. Anne, Behavioral based detection of android ransomware using machine learning techniques. *Int. J. Syst. Assur. Eng. Manag.* **15**(9), 4404–4425 (2024). <https://doi.org/10.1007/s13198-024-02439-z>
41. M. Aljabri, F. Alhaidari, A. Albuaainain, S. Alrashidi, J. Alansari, W. Alqahtani, J. Alshaya, Ransomware detection based on machine learning using

- memory features. *Egypt. Inform. J.* **25**, 100445 (2024). <https://doi.org/10.1016/j.eij.2024.100445>
42. K. Ganapathiyappan, A. Yadav, in *Cyber Warfare, Security and Space Computing, Communications in Computer and Information Science*, Optimized deep learning technique for the effective detection of windows pe malware, vol. 2195 (2025), pp. 359–370. https://doi.org/10.1007/978-3-031-73494-6_27
 43. G. Kirubavathi, W.R. Anne, U.K. Sridevi, A recent review of ransomware attacks on healthcare industries. *Int. J. Syst. Assur. Eng. Manag.* **15**(11), 5078–5096 (2024). <https://doi.org/10.1007/s13198-024-02496-4>
 44. P. Kothamali, S. Banik, Limitations of signature-based threat detection. *Int. J. Comput. Appl.* (2025). https://www.researchgate.net/publication/388494583_Limitations_of_Signature-Based_Threat_Detection. Accessed 29 May 2025
 45. G. Kim, Byte frequency based indicators for crypto-ransomware. *J. Comput. Sci. Technol.* **37**(2), 263–278 (2022). <https://doi.org/10.1007/s11390-022-0263-4>
 46. S. McIntosh, J. Jang-Jaccard, P. Watters, T. Susnjak, The inadequacy of entropy-based ransomware detection. *J. Cyber Secur. Technol.* (2023). https://www.researchgate.net/publication/337776801_The_Inadequacy_of_Entropy-Based_Ransomware_Detection. Accessed 29 May 2025
 47. R. Van Sloun, B. Erb, D. Erb, M. Hermann, L. Nussbaum, in *Network and Distributed System Security Symposium (NDSS)*, Detecting ransomware despite i/o overhead: A practical multi-staged approach (Internet Society, 2025). <https://www.ndss-symposium.org/wp-content/uploads/2025-764-paper.pdf>. Accessed 29 May 2025
 48. F. Casino, C. Patsakis, E. Batista, M. Fayyaz, P. Garcia, Not on my watch: ransomware detection through classification of encrypted streams. *J. Cybersecur.* **11**(1), tyaf009 (2025). <https://doi.org/10.1093/cybsec/tyaf009>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.